

Cahier des Charges : Résolution du Problème NP-Difficile de Kemeny.

Goales Saphiya

Teibi Zahra

Introduction

La théorie du vote s'intéresse aux règles qui permettent de transformer les préférences individuelles en une décision commune. Parmi les différentes méthodes existantes, l'agrégation de classements constitue un problème fondamental au cœur de la théorie du choix social, avec des applications allant de l'apprentissage automatique à la recherche web.

Objectif : établir un consensus à partir d'un ensemble de classements individuels.

Parmi les règles de vote, la règle de Kemeny :

- est l'une des plus pertinentes
- établit un classement de consensus qui minimise la distance totale de Kendall-tau (le nombre total de désaccords (paires inversées) entre le consensus et les votes individuels)

Elle est particulièrement reconnue pour ses propriétés axiomatiques robustes. Elle est la seule à satisfaire simultanément :

- le critère de Condorcet
- l'axiome de renforcement (cohérence lors de la fusion de groupes de votants)
- la neutralité (équité totale entre les candidats)

Néanmoins, cette solidité théorique est confrontée à une difficulté pratique majeure : sa complexité. En effet, l'un des principaux inconvénients de la règle de Kemeny est son coût computationnel : le calcul d'un classement de Kemeny optimal est un problème NP-difficile. Ce résultat est valable même lorsque le nombre de votants est petit.

Problématique : face à cette complexité, ce projet vise à implémenter des approches permettant de résoudre le problème de Kemeny de manière exacte et efficace, même sur des instances de grande taille. Nous implémenterons les différentes méthodes de décomposition et de réduction de données, puis, les algorithmes de résolutions sur les instances réduites.

Contexte et objectifs

L'objectif de ce projet est alors de comparer les méthodes de résolution (programmation dynamique et programmation linéaire) ainsi que les méthodes de décomposition (le critère de Condorcet étendu et la règle de majorité des 3/4). Afin d'obtenir des résultats d'évaluation réalistes, nous évaluerons ces méthodes sur des instances réelles stockées dans la bibliothèque Preflib. De plus, ce projet se concentre principalement sur des préférences complètes et sans égalité.

La réduction de données vise à identifier et à fixer des relations d'ordre entre candidats qui sont garanties d'apparaître dans toute solution optimale. Cette réduction constitue une phase de prétraitement essentielle pour rendre la résolution d'instances de grande taille abordable.

La règle de majorité des 3/4

La règle de majorité des 3/4 repose sur la notion de préférence forte et permet d'identifier des candidats dont la position relative peut être déterminée sans ambiguïté.

Un candidat a est sale s'il existe au moins un autre candidat b tel que :

- ni $a \succ_{3/4} b$
- ni $b \succ_{3/4} a$

Un candidat propre est un candidat non sale.

Les candidats sales nécessitent un traitement plus approfondi, car leur position relative reste incertaine même après application de la règle de majorité. Le classement final de Kemeny s'obtient en concaténant les classements résolus des sous-instances des candidats sales et des candidats dont la position est certaine.

Le critère de Condorcet étendu

Le critère de Condorcet étendu constitue une version plus robuste du critère de Condorcet classique, qui se concentre sur un unique vainqueur.

Le principe est le suivant : si un sous-ensemble de candidat C_α est majoritairement préféré à tous les candidats de l'ensemble complémentaire $C_\beta = C \setminus C_\alpha$ alors pour tout $x \in C_\alpha$ et $y \in C_\beta$, x est préféré à y par plus de la moitié des électeurs. Ainsi cet ensemble C_α doit se placer en tête de tout classement optimal P^* .

Ce principe peut être généralisé avec plusieurs sous-ensembles formant une partition de l'ensemble des candidats C .

Le classement de Kemeny sur C peut être obtenu en juxtaposant les classements de Kemeny trouvés sur les sous-ensembles C_α d'une partition.

Algorithme du critère de Condorcet étendu :

1. construction du graphe de majorité stricte
2. analyse de cycles dans ce graphe (identifier les incohérences de préférences)
3. ordonnancement des composantes fortement connexes

Programmation Dynamique

L'algorithme de programmation dynamique calcule le score de Kemeny (la somme des distances de Kendall-Tau) pour chaque sous-ensemble possible de candidats $C' \subseteq C$.

Pour les sous-ensembles C' de taille 1, le calcul d'un classement de Kemeny est trivial. Il y a un seul candidat c et le score de Kemeny associé est de 0.

Pour les sous-ensembles C' de taille $|C'| > 1$, l'algorithme teste itérativement chaque candidat $c \in C'$ comme étant celui placé en première position du classement local. Le candidat c qui minimise le score total, est donc placé en première position dans le classement optimal pour ce sous-ensemble C' .

Programmation linéaire

L'agrégation de classement de Kemeny nécessite que le classement final soit un ordre strict. Ce problème peut donc être modélisé par le programme linéaire en nombres entiers suivant :

$$\min \sum_{a,b} Q_{ab} \cdot x_{ba} + Q_{ba} \cdot x_{ab} \quad (1)$$

$$x_{ab} + x_{ba} = 1 \quad (2)$$

$$x_{ab} + x_{bc} + x_{ca} \geq 1 \quad (3)$$

$$x_{ab} \in \{0, 1\} \quad (4)$$

Ce programme linéaire vise à minimiser le score total de Kemeny et retourne une solution du problème d'agrégation de Kemeny .

Le modèle utilise les variables binaires $x_{ab} \in \{0, 1\}$ qui valent 1 si le candidat a est classé avant le candidat b dans le classement de Kemeny et les variables Q_{ab} représentant le nombre de fois où le candidat a est classé avant b dans le profil de préférences.

Les contraintes suivantes garantissent que les solutions x_{ab} représentent un ordre linéaire valide :

- Contraintes d'ordre strict (contrainte (2)) : cette contrainte assure que pour toute paire de candidats (a, b) , on ne peut pas avoir à la fois $a \succ b$ et $b \succ a$ dans le classement final.
- Contraintes de transitivité (contrainte (3)) : cette contrainte est essentielle car elle évite l'existence de cycle dans le classement.

Implémentation

Nous allons développer ce projet en python. Lien du git où est disponible le projet : [Projet](#)

Nous devons définir une classe *Instance* afin d'y stocker l'instance sur laquelle nous évaluons nos méthodes. Celle-ci sera composée des attributs suivants :

- `nb_votants` : correspondant au nombre de votants pour une instance
- `nb_candidats` : correspondant au nombre de candidats pour une instance
- `matPref` : une matrice qui compte les préférences. `matPref[i, j]` correspond au nombre de votants qui préfèrent i à j .
- `candidats` : un dictionnaire associant pour chaque candidat son numéro à son nom

L'attribut `matPref` servira à déterminer si un candidat est propre, calculer le score de Kemeny, et construire graphe de majorité, etc.

Pour initialiser l'*Instance* nous devons définir la méthode *lecture_fichier* qui lit un fichier `.soc` contenant une instance complète et sans égalité (les données réelles de Preflib) afin d'initialiser tous les attributs.

Pour procéder à la décomposition de l'instance, on aura également besoin des méthodes *majorite_trois_quart* et *critere_condorcet_etendu*. De même, pour les méthodes de résolutions exactes, nous aurons besoin des méthodes *resolution_PD* et *resolution_PL*.

Afin de construire le graphe de majorité pour le critère de Condorcet étendu, nous implémenterons la méthode *construction_graphe_majorite* qui construit une instance de la classe *Graphe*, ajoute un nœud par candidat et un arc entre deux candidats a et b si a est préféré par plus de 50% des votants à b .

La classe *Graphe* contiendra les attributs et méthodes suivantes :

attributs

- `noeuds` : ensemble de noeuds
- `voisins` : un dictionnaire d'adjacence

methodes

- `__str__` : une méthode d'affichage du graphe sur le terminal

- `afficher_graphe` : une méthode d’affichage du graphe plus visuel avec matplotlib
- `getNoeuds` : retourne l’ensemble des noeuds
- `addNoeud` : ajoute un noeud au graphe
- `addVoisins` : ajoute un voisin à un noeud
- `getVoisins` : retourne les voisins d’un noeud
- `get_CFC` : retourne les composantes fortement connexes du graphe à l’aide de l’algorithme de Tarjan. Cette fonction sera utile pour le critère de condorcet étendu.

Pour finir, nous implémenterons des méthodes d’évaluation telles que *evaluation_temps_calcul* qui estimera le temps d’exécution des différentes méthodes de résolutions (avec et sans décomposition). Nous aurons également une méthode *evaluation_decomposition* qui estimera la capacité des méthodes à décomposer l’instance en sous-instances, c’est-à-dire la taille de la plus grande sous-instance obtenue après la réduction.

Références